A PROJECT REPORT
ON
"SMART LIBRARY MANAGEMENT SYSTEM (SLMS)"
Paper code BCA-605
Submitted to

Ch. Charan Singh University, Meerut
In Partial Fulfilment for the Award of

B.C.A (BACHELOR'S OF COMPUTER APPLICATION)
At

Sunder Deep College of Management & Technology, Dasna Ghaziabad
(Session 2025–2026)

Under The Guidance of :

Ms. Arjoo Jain

Submitted By:

Kunal Panchal (Roll No :- 230302010367)
Aditya Pal (Roll No :- 230302010307)

Project Report Structure

i. Cover page/ Title of the project
ii. Project Category:
Application/RDBMS based project/Web based application/other
iii. Acknowledgement
iv. Student declaration
v. Certificate from the supervisor
vi. Table of Contents

Phase I

1. **About The Project**
   i. Introduction
   ii. Objective
2. **Preliminary Investigation**
   i. Tools/Platform used
   ii. Hardware /Software Requirement
3. **Feasibility Study**
   i. Operational Feasibility
   ii. Technical Feasibility
   iii. Economic Feasibility
4. **Information Gathering (Mention tools used for information Gathering)**
   i. On site Observation
   ii. Questionnaire
   iii. Interview
5. **Requirement Analysis & Specification (Final conclusion About the Information Gathering)**
   i. Data Flow Diagram (DFD)
6. **Logical Design**
   i. Entity Relationship Diagram (ERD)
   ii. Data Dictionary
   iii. Database/Table Design
   iv. PERT/GANTT Chart
   (As per the Project Requirement)
7. **Physical Design**
   i. Output Requirement
   • User Interface Design
   • Output Report Design
   ii. Input Requirement

Phase II

8. Process Design
i. Coding
ii. Constraints/Check

9. **Testing**
10. **Maintenance**
11. **Screen Shots**
    i. Forms
    ii. Reports
12. **Limitations/Future Scope**
13. **References/Bibliography**

PROJECT CATEGORY

This project falls under the Web Based Application / RDBMS Based Project category. It involves designing and developing "Smart Library Management System (SLMS)", a complete MERN-stack based library automation system for educational institutes and departmental libraries.

Unlike manual register-based library systems, this project digitizes and automates the processes of book management, user management, borrowing/returning, overdue tracking, fine calculation, and notification delivery in a structured and secure manner.

SLMS integrates:

• User Registration with Email OTP verification
• JWT-based Login using httpOnly cookies
• Role-based Access Control (Admin and Member)
• Book Management (Add/Edit/Delete/Search/Pagination)
• Book cover image upload using Cloudinary
• Borrow/Return workflow with due-date handling
• Automatic fine calculation (₹5/day overdue)
• Email notifications (Borrow confirmation, Return confirmation, Daily overdue reminders)
• Reports & analytics for Admin dashboard

The system stores all records in MongoDB database using Mongoose models, and provides REST APIs via Express.js. A modern React-based frontend provides user-friendly interaction for both Admin and Member users.

Repository Language Composition:
• JavaScript – 99.3%
• Other – 0.7%

STUDENT DECLARATION

We, solemnly affirm and declare that the project "**Smart Library Management System (SLMS)**" under the guidance of "**Ms. Arjoo Jain**" is our original work. No part of this work whether documentation and coding has been copied or taken in any form or by any means.

We certify that our work is original and it has been made as Major project held in BCA VI Semester

for the partial fulfillment the degree of "Bachelor of Computer Application".

Group members name with Roll no Signature

1. Kunal Panchal - 230302010367
2. Aditya Pal - 230302010307

ACKNOWLEDGEMENT

We take this opportunity to express our sincere thanks and deep gratitude to all those people who extended their whole hearted co-operation and have helped us in completing this project successfully.

We acknowledge the effort of those who have contributed significantly to our project. We express our sincere attitude and thank-fullness towards **Dr. Prof. Sandeep** (Director of SDEC) has provided such a talented faculty to us then We would like to thank **Mrs. Yogita Kaushik** (Head of the Department) for their guidance throughout BCA.

We sincerely thank to our guide "**Ms. Arjoo Jain**" for his inspiring guidance and constant motivation. Every time we were lurking in the dark, he showed us the way out.

We would like to express our gratitude to other faculty members of our college for their reviews and many helpful comments.

Finally, the project would not have been possible without confidence, endurance and support of our group members.

Name of Group Member

Kunal Panchal 230302010367
Aditya Pal 230302010307

CERTIFICATE FROM THE SUPERVISOR

This is to certify that **Kunal Panchal** and **Aditya Pal**, students of Sunder Deep College of Management & Technology, have successfully completed the major project titled "Smart Library Management System (SLMS)" under my supervision and guidance.

This project was carried out as part of the academic requirements for the award of the **Bachelor of Computer Applications (BCA)** degree during the academic session **2023–2026**.

The project involves the design and development of a full-stack **Smart Library Management System** built using the MERN stack. It demonstrates a comprehensive understanding of:

- Frontend development using React.js and Tailwind CSS
- Backend development using Node.js and Express.js
- Database design and integration using MongoDB and Mongoose
- User Authentication and Authorization (JWT, roles, cookies)
- Email OTP verification and automated email notification system
- Borrow/Return workflow and fine calculation logic
- Reports & analytics for administrative monitoring

Supervisor's Name: Ms. Arjoo Jain

Designation: Assistant Professor

Department: Computer Applications

Signature:

Date:

ABOUT THE PROJECT

INTRODUCTION

Smart Library Management System (SLMS) is a modern web-based solution developed to automate and manage day-to-day library activities. Traditional libraries often maintain book records, issue/return details, and fine calculations manually.

This creates problems such as data duplication, human errors, slow searching, and difficulty in tracking overdue books.

The goal of SLMS is to build a secure, scalable, and user-friendly system that provides:

- A centralized digital book catalog
- Secure user registration and login
- Role-based access for Admin and Member
- Book issuing and return tracking
- Automatic fine calculation
- Email-based notifications and reminders
- Reports and analytics for better management

This project is built using MERN Stack (MongoDB, Express.js, React.js, Node.js). MongoDB stores users, books, and borrow records.

The backend exposes REST APIs for frontend use. The frontend provides separate dashboards for Admin and Member users, improving usability.

OBJECTIVE

The main objective of the "**Smart Library Management System (SLMS)**" project is to design and develop a secure, efficient, and reliable web application that automates library operations, reduces manual work, ensures accurate record maintenance, and improves overall transparency.

Specific Objectives:

1. To implement secure user registration with Email OTP verification.
2. To provide JWT based login system with role-based authorization (Admin/Member).
3. To build a book catalog system with search, category filtering, and pagination.
4. To allow Admin to add, edit, delete books and upload book cover images.
5. To implement borrowing system with issue date and due date (14 days return window).
6. To implement return workflow with automatic fine calculation (₹5/day overdue).
7. To maintain complete borrowing history and overdue records for monitoring.
8. To send automated emails: OTP, borrow confirmation, return confirmation with fine details.
9. To schedule daily overdue reminder emails to reduce late returns.
10. To provide reports & analytics such as total books, users, borrows, fines, and most borrowed books.

PRELIMINARY INVESTIGATION

TOOLS/PLATFORM USED

The "**Smart Library Management System (SLMS)**" project was developed using full-stack web development tools and frameworks. Each tool played an important role in creating a secure and feature-rich system.

1. Core Technologies

• **MERN Stack:**
MERN stack provides a complete development environment for web applications. React is used for frontend UI, Node.js and Express are used for backend API services, and MongoDB is used for storing application data.

• **MongoDB (Database):**
Stores all entities such as Users, Books, Borrow Records, OTP verification status, and fine values.

• **Express.js (Backend Framework):**
Creates REST APIs for authentication, books, borrowing, and user management.

• **React.js (Frontend):**
Builds interactive and responsive user interface for Admin and Member dashboards.

2. Authentication & Security Technologies

• **JWT (JSON Web Token):**
Used for authentication and session management.

• **httpOnly Cookies:**
Used to store authentication tokens securely, reducing risk of XSS attacks.

• **bcryptjs:**
Used to hash and verify user passwords securely.

3. Email & Scheduling Tools

• **Nodemailer (Gmail SMTP):**
Used for sending OTP, borrow confirmation, return confirmation, and reminders.

• **node-cron:**
Used to schedule daily overdue reminder emails (e.g., every day at 9 AM).

4. Image Upload Tools

• **Cloudinary:**
Stores book cover images in the cloud.

• **express-fileupload:**
Uploads images from admin panel to backend.

5. Development Tools

• **Visual Studio Code:**
Used for coding and project management.

• **Postman:**
Used to test backend API endpoints.

• **Git & GitHub:**
Used for version control and repository management.
Repository Language Composition:

• JavaScript – 99.3%
• Other – 0.7%

HARDWARE / SOFTWARE REQUIREMENT

Since SLMS involves a full-stack web application with database and email services, moderate system requirements are sufficient.

a) Minimum Hardware Configuration

• Processor: Intel Core i3 / AMD Ryzen 3 or equivalent
• RAM: Minimum 4 GB (8 GB recommended)
• Hard Disk Space: At least 5 GB free
• Display: 1366 × 768 resolution
• Keyboard and Mouse: Required
• Internet Connection: Required for database and email services

b) Recommended Hardware Configuration

• Processor: Intel Core i5 / i7 or AMD Ryzen 5 / 7 series
• RAM: 8 GB or higher
• Storage: SSD recommended
• Display: 1920 × 1080 resolution
• Network: Stable internet connection

Software Requirements

• Operating System: Windows 10/11 (64-bit), Linux, or macOS
• Node.js v18 or higher
• npm/yarn
• MongoDB Atlas or local MongoDB
• Gmail account with App Password for SMTP
• Cloudinary account for image hosting

FEASIBILITY STUDY

i. OPERATIONAL FEASIBILITY

Operational feasibility determines whether the system can operate effectively in the real environment and satisfy user requirements.

SLMS is operationally feasible because:
• It provides quick search and access to book details.
• It reduces manual data entry and errors by maintaining digital records.
• It supports both Admin and Member roles with proper access control.
• It automatically calculates fines, reducing disputes and mistakes.
• It notifies users via email, improving transparency and timely returns.
• It provides reports and analytics for administrative decisions.

From the library perspective, this system reduces staff workload and improves reliability of records.

ii. TECHNICAL FEASIBILITY

Technical feasibility checks whether the system can be built with available technologies and resources.

SLMS is technically feasible because:
• MERN stack is widely used and well-documented.
• MongoDB provides flexible storage for library records.
• JWT and cookies provide secure authentication.
• Nodemailer provides reliable email communication.
• node-cron enables automated reminders without manual effort.
• Cloudinary provides scalable image storage for book covers.

The system architecture supports scalability and modular enhancement.

iii. ECONOMIC FEASIBILITY

Economic feasibility (cost-benefit analysis) ensures that the project can be developed within acceptable cost.

SLMS is economically feasible because:

• It uses open-source technologies (React, Node, Express, MongoDB).
• MongoDB Atlas and Cloudinary provide free tiers for academic usage.
• No licensing cost is required.

INFORMATION GATHERING (MENTION TOOLS USED FOR INFORMATION GATHERING)

ON SITE OBSERVATION

On-site observation was conducted by studying common library workflows used in manual systems:

• Book entry and catalog maintenance
• Issue process using manual register entries
• Return process with manual due date checking
• Fine calculation by counting overdue days
• Preparation of overdue lists manually

Problems observed:

• Slow book search and availability checking
• Human errors in entry and fine calculation
• Difficulty in creating reports quickly
• Lack of automated reminder mechanism
• No centralized system for borrow history tracking

Tools used for observation and study:

• Direct workflow observation
• Notes documentation in Word/Notebook
• Review of similar systems and online documentation

QUESTIONNAIRE

A questionnaire was used to gather feedback and requirements from potential users.
Purpose of questionnaire:

• To identify essential features of a library system
• To understand user expectations for search, borrowing, and fine management
• To validate the need for OTP verification and notifications
• To confirm the importance of reports for admin

Sample questions:

1. Should registration require OTP verification?
2. What search filters are needed (title, author, category)?
3. Should the system calculate fine automatically?
4. Should users view their borrowing history?
5. Should the system send overdue reminders by email?
6. What reports are required for admin?

Tools used:

• Google Forms / manual questionnaires
• Google Sheets / Excel analysis

Key findings:

• Users prefer fast search and clear availability status.
• OTP verification is useful to avoid fake accounts.
• Automatic fine calculation is essential.
• Overdue reminders reduce late returns.
• Admin reports are necessary.

INTERVIEW

Interviews were conducted with peers and academic users for deeper understanding.

Interview focus:

- Borrow period recommendation
- Fine rule preference
- Need for role-based control
- Importance of notifications and reports

Key conclusions from interviews:

- Borrow period: 14 days
- Fine rule: ₹5/day overdue
- Admin should manage roles and books
- Daily overdue reminders should be implemented.

REQUIREMENT ANALYSIS & SPECIFICATION (FINAL CONCLUSION ABOUT INFORMATION GATHERING)

After analyzing observation, questionnaire, and interview results, the requirements were finalized.

Functional Requirements:

a) The system must allow users to register and verify email using OTP.
b) The system must authenticate users using JWT and store tokens securely in cookies.
c) The system must support role-based access: Admin and Member.
d) Admin must be able to add, edit, and delete books.
e) Books must support search and pagination.
f) The system must allow issuing of books with due date generation.
g) The system must allow returning of books with automatic fine calculation.
h) The system must maintain borrow history records.
i) The system must send emails for OTP, borrow confirmation, return confirmation, and overdue reminders.
j) The system must provide admin reports and analytics.

Non-Functional Requirements:

a) Security: passwords hashed, JWT secure, role checks.
b) Performance: search and pagination must be efficient.
c) Reliability: data consistency in issue/return actions.
d) Usability: easy to use interface.
e) Maintainability: modular code structure.

i. DATA FLOW DIAGRAM (DFD)

Level 0 (Context Diagram):

External Entities:
• Admin
• Member
• Database (MongoDB)
• Email Service (SMTP)
• Cloudinary

Flow:

1. Admin/Member interacts with SLMS via UI.
2. SLMS reads/writes data to MongoDB.
3. SLMS sends OTP/notifications to users through Email Service.
4. SLMS uploads and fetches book cover images using Cloudinary.

Level 1 (Detailed DFD):

1. Authentication Process
   • Register user → Send OTP → Verify OTP → Login → Session created
2. Book Management Process
   • Add/Edit/Delete book → Store in database → Display catalog
3. Borrow/Return Process
   • Issue book → Create borrow record → Set due date → Update availability
   • Return book → Calculate fine → Update record → Send email confirmation
4. Reports Process
   • Generate totals (books, users, borrows)
   • Generate overdue list
   • Generate fine summary and most borrowed books list

LOGICAL DESIGN

ENTITY RELATIONSHIP DIAGRAM (ERD)

Entities:

1. User
2. Book
3. BorrowRecord

Relationships:
• User (1) to BorrowRecord (Many)

One user can have multiple borrow records.
• Book (1) to BorrowRecord (Many)

One book can appear in multiple borrow records over time.
BorrowRecord acts as a relationship entity containing issue and return details.

DATA DICTIONARY

User Entity: • _id: ObjectId (Primary Key)
• name: String
• email: String (unique)
• password: String (hashed)
• role: String (admin/member)
• isVerified: Boolean
• createdAt: DateTime

Book Entity: • _id: ObjectId (Primary Key)
• title: String
• author: String
• category: String
• quantity: Number
• coverImage: String (Cloudinary URL)
• createdAt: DateTime

BorrowRecord Entity: • _id: ObjectId (Primary Key)
• userId: ObjectId (Reference User)
• bookId: ObjectId (Reference Book)
• issueDate: Date
• dueDate: Date
• returnDate: Date (nullable)

• fine: Number
• status: issued/returned/overdue

DATABASE/TABLE DESIGN

Database: MongoDB (NoSQL)

Collections:
• users
• books
• borrows

Indexes (recommended): • users.email (unique)
• books.title (search index)
• borrows.userId, borrows.bookId (query optimization)

PERT/GANTT CHART

PERT (activity sequence):

1. Requirement gathering
2. System design
3. Database design
4. Backend API development
5. Frontend UI development
6. Integration and testing
7. Deployment preparation
8. Documentation and final review

Gantt chart (timeline representation example):
Week 1: Requirement + design
Week 2: Backend + database
Week 3: Frontend + integration
Week 4: Testing + documentation

PHYSICAL DESIGN

OUTPUT REQUIREMENT

• User Interface Design

Admin UI outputs:
• Admin Dashboard (total books/users/borrows, fine summary)
• Book management pages (add/edit/delete books)
• Borrow records page (issued, returned, overdue)
• User management page (role update)
• Reports pages (overdue list, fine summary, most borrowed books)

Member UI outputs:
• Book catalog with search and pagination
• My borrow records page
• Profile page
• Notifications received by email
• Output Report Design

Reports generated by system:
• Overdue books report
• Fine collection summary report
• Most borrowed books list report
• Borrow history report

INPUT REQUIREMENT

Inputs include:
• Registration details (name, email, password)
• OTP code for verification
• Login credentials
• Admin book details (title, author, category, quantity, cover image)
• Borrow issue and return requests
• User role update by admin

PROCESS DESIGN

CODING (PROCESS FLOW)

Module 1: Authentication
• Register user → OTP email sent → User verifies OTP → Account verified
• Login → JWT generated → Stored in httpOnly cookie
• Role-based route protection applied

Module 2: Book Management
• Admin adds book → cover image uploaded to Cloudinary → data stored in MongoDB
• Admin edits/deletes books
• Users search books using title/author/category and pagination

Module 3: Borrowing
• Issue book → Borrow record created with issue date and due date (14 days)
• Book availability updated
• Borrow confirmation email sent

Module 4: Returning
• Return book → Fine calculated automatically if overdue
• Record updated with return date and fine
• Return confirmation email sent with fine details

Module 5: Overdue Reminder
• Cron job runs daily
• Finds overdue borrow records
• Sends reminder emails to users

Module 6: Reports & Analytics
• Calculates total books, users, borrows
• Generates fine summary and most borrowed list
• Displays overdue records

API Endpoints (As per implementation):

Authentication:
• POST /api/auth/register
• POST /api/auth/verify-otp
• POST /api/auth/resend-otp
• POST /api/auth/login

• GET /api/auth/logout
• POST /api/auth/forgot-password
• PUT /api/auth/reset-password/:token
• GET /api/auth/me

Books:
• GET /api/books
• GET /api/books/:id
• POST /api/books (Admin)
• PUT /api/books/:id (Admin)
• DELETE /api/books/:id (Admin)

Borrowing:
• POST /api/borrow/issue
• PUT /api/borrow/return/:id
• GET /api/borrow/records (Admin)
• GET /api/borrow/my-records (Member)
• GET /api/borrow/overdue (Admin)

Users:
• GET /api/users (Admin)
• GET /api/users/:id (Admin)
• PUT /api/users/:id/role (Admin)
• DELETE /api/users/:id (Admin)

CONSTRAINTS/CHECK

• Only verified users can login.
• Only Admin can add/edit/delete books and manage users.
• Book cannot be issued if quantity is not available.
• Due date is calculated based on issue date (14 days).
• Fine is calculated only when return date exceeds due date.
• Token must be validated on protected routes.
• Email sending must handle errors and retries if needed.

TESTING

Testing ensures the correctness and reliability of the SLMS system.

i. Test Environment

• Frontend: React + Vite
• Backend: Node.js + Express.js
• Database: MongoDB Atlas
• Tools: Postman, browser, console logs

ii. Functional Testing

Module Test Performed Expected Output Result
Registration Register user OTP email sent Pass
OTP Verification Verify OTP Account verified Pass
Login Login user JWT cookie created Pass
Book CRUD (Admin) Add/Edit/Delete book Data updated in DB Pass
Book Search Search by title/author/category Correct filtered results Pass
Borrow Issue Issue a book Borrow record created, due date set Pass
Return Book Return book Fine calculated if overdue Pass
Email Notifications OTP/borrow/return emails Email delivered Pass
Overdue Reminder Cron Daily reminder email Overdue users notified Pass

iii. Non-Functional Testing

• Security Testing: password hashing, JWT validation, role protection
• Performance Testing: pagination and search improves performance
• Reliability Testing: multiple issue/return operations behave consistently

MAINTENANCE

The maintenance phase ensures that SLMS continues to operate reliably after development.

1. Corrective Maintenance
   - Fix issues in email delivery or OTP verification
   - Fix bugs in borrow/return logic and fine calculation
   - Fix UI problems and validation issues
2. Preventive Maintenance
   - Regular dependency updates for security
   - Database backup and monitoring
   - Log monitoring for cron jobs and email sending
3. Adaptive Maintenance
   - Add new features based on feedback (reservation, barcode scanning)
   - Improve compatibility with different deployment environments
4. Perfective Maintenance
   - UI/UX enhancements
   - Advanced reports and analytics

SCREENSHOTS

i. Forms (Screens)
• Home Page
• Register Page
• OTP Verification Page
• Login Page
• Forgot Password Page
• Reset Password Page
ii. Reports / Dashboards
• Admin Dashboard
• Manage Books
• Add Book Form
• Manage Users
• Borrow Records
• Overdue Records
• Reports Page (Fine summary, Most borrowed books)
• Member Dashboard
• Book Catalog
• My Borrow Records

LIMITATIONS / FUTURE SCOPE

Limitations

• System depends on internet for Email SMTP and Cloudinary services.
• Barcode/RFID scanning is not included.
• Payment gateway for fine collection is not implemented.
• Email delivery depends on SMTP provider availability.

Future Scope

• Barcode/QR based issuing and returning system.
• SMS/WhatsApp notification integration.
• Online fine payment gateway integration.
• Book reservation and waitlist facility.
• Multi-branch library support and centralized monitoring.
• Advanced analytics dashboard using graphs and charts.

REFERENCES/BIBLIOGRAPHY

The following references and resources were used during the research, design, and development of **Smart Library Management System (SLMS)**:

1. Official Documentation

a. React Documentation — https://react.dev/
b. Vite Documentation — https://vitejs.dev/guide/
c. Node.js Documentation — https://nodejs.org/en/docs
d. Express.js Documentation — https://expressjs.com/
e. MongoDB Documentation — https://www.mongodb.com/docs/
f. Mongoose Documentation — https://mongoosejs.com/docs/
g. JSON Web Token (JWT) Documentation — https://jwt.io/introduction
h. Nodemailer Documentation — https://nodemailer.com/about/
i. Cloudinary Documentation — https://cloudinary.com/documentation
j. node-cron Documentation — https://www.npmjs.com/package/node-cron

2. Books and Academic References

a. Database System Concepts — Abraham Silberschatz, Henry F. Korth, S. Sudarshan
b. Web Technologies: A Computer Science Perspective — Jeffrey C. Jackson
c. Software Engineering — Ian Sommerville
d. Operating System Concepts — Abraham Silberschatz, Peter B. Galvin, Greg Gagne (for understanding system-level scheduling concepts used in cron-based reminders)

3. Online Technical Resources

a. MongoDB Atlas Documentation — https://www.mongodb.com/atlas/database
b. MDN Web Docs (JavaScript, HTTP, Cookies, Security) — https://developer.mozilla.org/
c. OWASP Web Security Testing Guide — https://owasp.org/www-project-web-security-testing-guide/
d. OWASP Top 10 (Web Application Security Risks) — https://owasp.org/www-project-top-ten/
e. GitHub Repository (Project Source) — https://github.com/KunalDev69/smart-library-management-system

4. Development Tools Used

a. Visual Studio Code — Code editing and project management
b. Git & GitHub — Version control and project tracking
c. Postman — API testing and debugging
d. MongoDB Compass — Database visualization and query testing
e. Node Package Manager (npm) — Dependency management and build scripts
f. Chrome DevTools — Frontend debugging and network inspection

Conclusion

This documentation summarizes the complete lifecycle of the **Smart Library Management System (SLMS)** — from requirement analysis and system design to full-stack development, testing, and maintenance.